

INFORME EVOLUTIVO · CONFIDENCIAL

Historia evolutiva del proyecto: de 7 empresas a una base de datos financiera

Proyecto	Módulo de Análisis Financiero — Backoffice Catalaxia
Cliente	Catalaxia S.A.
Fecha	Junio 2026

Para quién es esta guía: un developer full-stack que sabe programar pero **no sabe nada de finanzas**, y que se suma al proyecto ahora. El objetivo no es que aprendas SEC EDGAR de cero (para eso ya existe [GUIA_SEC_EDGAR_PARA_DEVS.md](#) — lee primero si no sabés qué es un CIK, un ratio o un 10-K). **Esta guía cuenta otra cosa: la historia de cómo se construyó el proyecto**, en qué orden, qué problemas aparecieron, cómo se resolvieron, y por qué el alcance fue creciendo en cada etapa hasta llegar a donde está hoy.

Está escrita para leerse **de arriba hacia abajo, en orden cronológico**. Cada fase existe porque la fase anterior, aunque funcionó, no alcanzaba.

0. El problema que arrancó todo

Catalaxia (el cliente) necesita un **screener financiero**: una tabla con cientos de empresas (CEDEARs — acciones de EE.UU. que cotizan en Argentina) donde se puedan comparar ratios como el PER, el margen neto o el ROE para decidir si una acción está cara o barata.

Para llenar esa tabla hacían falta **datos financieros confiables**. Había dos caminos:

- ❌ **Scraping a Investing.com**: es lo que se probó primero y **falló**. Quedó documentado en [failed_investing_scraping/README.md](#) junto con 9 scripts de prueba ([demo_scraping_*.py](#) , [demo_inspect_*.py](#) , etc.). Es frágil, viola los Términos de Servicio de Investing, y el sitio detecta y bloquea bots.
- ✅ **SEC EDGAR**: la base de datos oficial del gobierno de EE.UU. Gratis, sin scraping, legal. El problema: los datos vienen **crudos** (no hay un campo "PER" listo para usar, hay que calcularlo desde cero).

La decisión fue ir con EDGAR. Pero surgió la pregunta obvia: **¿de verdad se puede reconstruir lo que muestra Investing.com calculando todo a mano desde datos crudos?** Nadie lo sabía todavía. De ahí nace la Fase 1.

Fase 1 — ¿Esto funciona? (5 empresas USA + 2 ADRs argentinos)

Objetivo de esta fase: validar la hipótesis antes de construir nada grande.

Qué se hizo: se tomaron 5 empresas estadounidenses muy conocidas (AAPL, MSFT, NVDA, TSLA, AMZN) más 2 ADRs argentinos, se calcularon a mano los ratios desde los datos crudos de EDGAR, y se compararon contra lo que mostraba Investing.com para esas mismas empresas.

Qué se encontró (documentado en [docs/calculos-financieros/documentacion/RESUMEN_FINAL_HALLAZGOS.md](#)):

Dato	Resultado
Revenue, Net Income	convergen casi exacto (<1% y <0,1% de diferencia)
Deuda/Equity	converge bien (<4%)
P/E ratio y EBITDA pre-calculados	divergen fuerte — hasta 54,9% en NVDA

El aprendizaje clave de esta fase: Investing.com usa **GAAP puro** (el estándar contable oficial) para las cosas importantes, no los números "ajustados" (non-GAAP) que algunas empresas prefieren mostrar en sus propios reportes para parecer más rentables. Esto se documentó en [GUIA_RATIOS_EDGAR_vs_INVESTING.md](#): **qué tomar directo de**

EDGAR, qué calcular a mano, y qué no usar nunca (ver `EJEMPLO_CALCULO_LOCAL.md` para el cálculo completo de AAPL paso a paso).

Conclusión de la Fase 1: "esto funciona, EDGAR puede reemplazar a Investing.com" — pero probado sobre 7 empresas, todas grandes y conocidas. La pregunta que quedó abierta: **¿funciona igual con un universo más realista, mezclando empresas chicas, bancos, y compañías extranjeras?** De ahí nace la Fase 2.

 **Nota de alcance:** acá el plan original (`PLAN_SCREENERS_218_ACCIONES.md`) apuntaba a 218 acciones (200 CEDEARs + 18 ADRs). Ese plan **nunca se llegó a ejecutar como estaba escrito** — quedó superado por el camino real que siguió el proyecto (Fases 2 y 3). Es interesante leerlo como referencia de "qué se pensaba al principio", no como lo que pasó.

Fase 2 — Validar a escala real: 99 tickers (`Tickets.xlsx`)

Objetivo de esta fase: dejar de probar con 7 empresas "fáciles" y probar con una lista real de 99 tickers, mezclando blue chips de EE.UU. con ADRs extranjeros difíciles (India, Japón, Brasil, Rusia, Taiwán...).

Esta es la fase donde aparecieron **todos los problemas reales** que una muestra de 7 empresas conocidas nunca iba a mostrar. El pipeline quedó en `scripts/tickets/` (`01_mapear_cik.py` → `07_generar_estadisticas.py`), cada script resuelve un paso:

Paso 1 — ¿Cuál es el CIK de cada ticker?

Antes de pedirle nada a EDGAR hay que traducir el ticker (`AAPL`) a su CIK (`0000320193` , el ID que usa la SEC). Se construyó `01_mapear_cik.py` usando el archivo oficial `company_tickers.json` de la SEC.

Problema encontrado: dos tickers (`GOLD` y `CHA`) **ya no representan lo que uno esperaría**. La SEC reasigna tickers cuando una empresa cambia de nombre o se deslista: `GOLD` hoy es "Gold.com, Inc." (no Barrick, que pasó a cotizar como `B`), y `CHA` hoy es una cadena china de té (no China Telecom, deslistada en 2021).

Cómo se resolvió: en vez de adivinar o usar el dato "casi seguro correcto", se **excluyeron explícitamente** y se mandaron a una lista de revisión manual. Resultado: 99 → 77 tickers con CIK resuelto con confianza.

Paso 2 — Descargar financieros (EDGAR) y precios (yfinance)

`02_descargar_datos.py` reusa los fetchers de `backend/fetchers/` para bajar, por cada CIK, todo el historial financiero, y por cada ticker, el precio actual.

Problema encontrado: la librería `yfinance` , en la versión que estaba fijada en `backend/requirements.txt` (`0.2.50`), **ya no funcionaba** — Yahoo Finance había cambiado su API y la librería vieja no sabía hablarle más. Fallaban los 77 tickers, el 100%.

Cómo se resolvió: actualizar `yfinance` a una versión nueva (`1.4.1`) resolvió todo de inmediato. Lección: una dependencia externa "que siempre funcionó" puede romperse sin avisar si el proveedor (Yahoo, en este caso) cambia algo del otro lado.

Resultado de este paso: 77/77 precios OK. De los 77 financieros, **65 bajaron bien** y **12 fallaron** — todos emisores extranjeros sin datos completos en `us-gaap` (ver el problema de IFRS más abajo).

Paso 3 — Calcular los ratios (acá aparecieron los bugs más interesantes)

`03_calcular_ratios.py` toma los datos crudos y aplica las fórmulas de la Fase 1 (TTM, márgenes, ROE, etc.).

Bug #1 — CAT (Caterpillar) con un PER de 106. Investigando por qué el margen neto y el PER de Caterpillar salían absurdos, se encontró que el tag `NetIncomeLoss` de esa empresa **deja de tener datos desde 2011** — Caterpillar consolida una subsidiaria financiera y, por eso, taguea su ganancia neta con `ProfitLoss` en vez de `NetIncomeLoss`. El cálculo estaba dividiendo un Net Income de hace 15 años contra acciones de hoy.

Cómo se resolvió: agregar `ProfitLoss` como alternativa (*fallback*) de `NetIncomeLoss` — un cambio de una línea, ya documentado en `GUIA_SEC_EDGAR_PARA_DEVS.md` pero que **nadie había implementado todavía**.

Bug #2 — los datos IFRS nunca se extraían (encontrado, pero NO arreglado en esta fase). Las empresas extranjeras (India, Brasil, Europa) no usan la taxonomía `us-gaap`, usan `ifrs-full`, que tiene **otros nombres de tag** (`ProfitLoss` en vez de `NetIncomeLoss`, `Revenue` en vez de `Revenues`, etc.). El código que hace el fallback a `ifrs-full` seguía buscando los nombres de `us-gaap` — entonces nunca encontraba nada, y esas 11 empresas quedaban marcadas como "sin datos" cuando en realidad **sí tenían datos, solo que con otro nombre**. Se verificó con ING (un banco holandés con 482 tags ifrs-full disponibles) que el dato realmente estaba ahí.

Por qué no se arregló en el momento: arreglarlo bien requería mapear ~10 tags IFRS y decidir qué hacer con monedas no-USD (EUR, GBP, etc.) — una decisión de diseño más grande de lo que daba el tiempo de esa sesión. Se documentó como pendiente en vez de improvisar una solución a medias. **Esto es importante: quedó anotado como deuda técnica consciente, no como un descuido.**

Resultado de este paso: de 65 empresas con financials, 48 quedaron con ratios completos. El resto, NULL explícito en lo que faltaba — nunca un número inventado.

Paso 4 y 5 — Reporte y comparación contra Investing.com

`04_generar_reporte.py` armó el primer Excel. `05_comparar_investing.py` cruzó esos resultados contra una planilla cargada a mano desde Investing.com (`informe_seguimiento_detallado.xlsx`).

Problema encontrado (de infraestructura, no de datos): el archivo CSV que se estaba editando se corrompió cuando Excel lo volvió a guardar — cambió el separador de columnas de `,` a `;` (por la configuración regional en español) y agregó cientos de filas vacías. El primer intento de sobrescribirlo **empeoró el daño**.

Cómo se resolvió: restaurar desde un backup hecho *antes* de tocar nada (por eso conviene **siempre** hacer una copia antes de escribir sobre un archivo real), y blindar el script para que: detecte el separador automáticamente, valide que el archivo tiene la forma esperada *antes* de escribir, y escriba primero a un archivo temporal que se valida antes de reemplazar el real.

Problema encontrado (conceptual): comparar el ROE de Investing.com contra nuestro cálculo daba resultados sin sentido. La razón: Investing.com muestra el ROE como una foto puntual (Net Income TTM / Equity), pero el pipeline solo tenía calculado el **CAGR del ROE a 5 años** (una tasa de crecimiento, no un nivel). Son dos magnitudes distintas — compararlas es como comparar "temperatura de hoy" contra "cuánto subió la temperatura en 5 años".

Cómo se resolvió: agregar el cálculo de ROE TTM simple, sin tocar el CAGR existente (que sirve para otra cosa).

Paso 6 y 7 — Mostrar la cuenta completa y encontrar outliers

`06_generar_calculos_edgar.py` agregó una solapa que desglosa **cada ratio en su fórmula completa** con celdas de Excel encadenadas (PER = Precio / EPS_TTM, donde EPS_TTM = Net Income TTM / Acciones, etc.) — para que cualquiera pueda auditar de dónde sale cada número sin abrir el JSON crudo.

`07_generar_estadisticas.py` calculó estadística (media, desvío, P50/P90/P95/P99) de la divergencia entre EDGAR e Investing, y para los **outliers** (los peores casos) hizo **ingeniería inversa**: "si el número de Investing fuera correcto, ¿qué Net Income/Equity/Dividendos implicaría? ¿Y qué tan lejos está eso de lo que realmente dice EDGAR?". Así se encontró, por ejemplo, que **NMR (Nomura)** tiene un error de escala **en el propio filing que la empresa le mandó a la SEC** (las acciones en circulación están tageadas con un millón de veces el valor real) — no es un bug nuestro, es un dato sucio en el origen.

El documento que junta todo esto es `DIAGNOSTICO_INVESTING_vs_EDGAR.md` : tiene las **10 causas raíz** encontradas en esta fase y el efecto medido de cada fix — las divergencias mayores al 10% bajaron de **107 a 39** (-64%) a medida que se iban resolviendo.

Conclusión de la Fase 2

El método funciona **muy bien** para empresas de EE.UU. (mediana de divergencia: PER 1,3%, EPS y Margen casi exactos) y **bastante mal** para empresas extranjeras — pero ya se sabe exactamente por qué (el bug de IFRS pendiente) y exactamente qué haría falta para arreglarlo.

Cómo cambió el alcance: se empezó queriendo "completar los ratios de Tickets.xlsx" y se terminó con un pipeline reproducible, 10 bugs documentados, y una lista clara de qué falta — mucho más valioso que los 99 números en sí.

Fase 3 — De 99 a 8.021: la escala real (trabajo en sesión paralela)

Importante para el lector nuevo: esta fase **no la hicimos en la misma sesión** que la Fase 2 — la hizo otra sesión de trabajo (vía la interfaz web), en paralelo. Lo que sabemos de esta fase viene de `BITACORA_Y_DECISIONES.md` y `ESPEC_TAGS_RATIOS.md`, más evidencia real de ejecución en `data/log_bloque1.txt`, `data/log_bloque2.txt` y `data/log_precios.txt` (esos logs muestran contadores reales de progreso, no son una afirmación sin respaldo).

La pregunta que motivó esta fase: la Fase 2 probó que el método funciona sobre 99 tickers elegidos a mano. Pero, **¿cuántas empresas hay en total disponibles en EDGAR, y hasta dónde se puede escalar esto?**

Lo que se midió: SEC EDGAR tiene **8.021 empresas con ticker**. De esas, ~82% reportan en `us-gaap`, ~4% en `ifrs-full`, el resto son fondos/ETFs sin estados financieros tradicionales. El **S&P 500 está 100% cubierto** — es el universo ideal para arrancar (empresas grandes, ~19 años de historia, máxima confiabilidad).

El fix de IFRS que había quedado pendiente — acá se resolvió

Esta fase retomó exactamente el bug que la Fase 2 dejó documentado y sin arreglar: se construyó un **mapeo canónico GAAP↔IFRS** (`NetIncome` viene de `NetIncomeLoss` o `ProfitLoss`, según la taxonomía; `Revenue` viene de `Revenues` o `Revenue`, etc.), con los nombres de tag IFRS descubiertos **empíricamente** probando contra ADRs reales (GGAL, ABEV, VIV, PAM, UGP, NU, GGB, LOMA). Esto es lo que finalmente desbloqueó a casi toda la plaza argentina y brasilera — sin este fix, quedaban afuera del dataset.

Arquitectura nueva: 5 bloques en vez de un script único

1. **Catálogo** — las 8.021 empresas con sector/país/tamaño.
2. **Facts** — todos los tags XBRL por empresa, GAAP+IFRS unificados.

- 3. **Ratios fundamentales** — ~28 ratios (márgenes, ROE, deuda, FCF, CAGR).
- 4. **Valuación** — PER, P/Book, EV/EBITDA, usando **market cap en USD + tipo de cambio** para las empresas que reportan en moneda local (evita el lío de mezclar un precio en dólares con un dato financiero en pesos/reales).
- 5. **Flags de calidad** — el cinturón de seguridad: en vez de borrar un dato raro, se marca (`ni_fy` , `roe_ns` , `fx` , `mktcap_rev`) y se deja a criterio humano.

Resultado medido

- **553 empresas** con ratios calculados (S&P 500 completo + ADRs Argentina/Brasil/LatAm), de las 8.021 totales.
- **429 de esas 553 (78%) quedan limpias** (sin ningún flag) — listas para screening serio sin revisión adicional.
- Validado de nuevo contra Investing.com sobre este dataset más grande: **PER mediana 1,3%, ROE mediana 0,4%** — coincide con lo que ya se había medido en la Fase 2, ahora confirmado a mayor escala.

Lo que todavía falta (pendiente, documentado, no resuelto)

- 1. Validar la valuación del universo nuevo contra Investing (recién se midió sobre los ratios fundamentales).
- 2. La "cola larga" de EE.UU. (~7.458 empresas más chicas que todavía no se bajaron).
- 3. Historia completa (hoy se guardan 6 años, se puede ampliar reprocesando el raw cache ya descargado, sin volver a pedirle nada a la SEC).
- 4. Cobertura de intereses y ROIC (faltan 2 conceptos en el mapeo).

El alcance, de punta a punta: cómo fue cambiando

	Fase 1	Fase 2	Fase 3
Empresas	7 (elegidas a mano, conocidas)	99 → 77 con CIK → 65 con datos	8.021 catalogadas → 553 con ratios
Pregunta que respondía	¿EDGAR puede reemplazar a Investing?	¿Funciona con un universo real y difícil?	¿Hasta dónde se puede escalar?
Taxonomía	us-gaap (no se tocó IFRS)	us-gaap + IFRS detectado roto, sin arreglar	us-gaap + IFRS arreglado
Resultado de confiabilidad	~99% en lo importante (5 USA)	PER 1,3%, pero solo en empresas USA	PER 1,3% confirmado a escala, 78% del dataset "limpio"
Qué falta al cerrar la fase	Probar a escala real	Arreglar IFRS, escalar más allá de 99	Cola larga USA, historia completa, valuación de ADRs

El patrón que se repite en las 3 fases: cada fase termina probando que el método anterior funcionaba, pero revela un límite nuevo que la fase siguiente ataca. Ninguna fase fue "la solución final" — cada una dejó la cancha más chica para la próxima.

Lecciones que se repiten en las 3 fases (las que importa recordar)

- **Nunca estimar un dato que falta.** Si un componente no está, el ratio queda `NULL`. Un `NULL` honesto es mejor que un número inventado que alguien podría confundir con un dato real.
- **"Oficial" no significa "sin errores".** El caso de Nomura (un error de escala de un millón de veces, en el propio filing que la empresa mandó a la SEC) demuestra que hay que validar incluso la fuente "de la verdad".
- **Comparar A contra B exige que A y B midan lo mismo.** El ROE TTM vs. CAGR y el EPS anual vs. TTM son los dos ejemplos reales de este proyecto: se ven parecidos, son cosas distintas, y confundirlos rompe cualquier conclusión.
- **Un bug que no se puede arreglar bien en el momento, se documenta como pendiente — no se improvisa una solución a medias.** El fix de IFRS es el ejemplo perfecto: se encontró en la Fase 2, se dejó anotado, y se resolvió recién en la Fase 3 cuando hubo tiempo de hacerlo bien.
- **Las dependencias externas se rompen sin avisar.** `yfinance` dejó de funcionar de un día para el otro porque Yahoo cambió algo de su lado. Fijar una versión no es "configurar una vez y olvidarse".
- **Hacer backup antes de escribir sobre un archivo real, siempre.** El incidente del CSV corrompido se resolvió en segundos porque existía una copia de antes.

Mapa rápido: dónde leer más de cada fase

Fase	Documentos clave
1	<code>docs/calculos-financieros/documentacion/RESUMEN_FINAL_HALLAZGOS.md</code> , <code>GUIA_RATIOS_EDGAR_vs_INVESTING.md</code> , <code>EJEMPLO_CALCULO_LOCAL.md</code>
2	<code>scripts/tickets/*.py</code> (el código), <code>docs/screener/DIAGNOSTICO_INVESTING_vs_EDGAR.md</code> (los 10 bugs), <code>informe_seguimiento_detallado.xlsx</code> (los datos en vivo)
3	<code>docs/screener/BITACORA_Y_DECISIONES.md</code> , <code>docs/screener/ESPEC_TAGS_RATIOS.md</code> , <code>data/log_bloque1.txt</code> / <code>log_bloque2.txt</code> / <code>log_precios.txt</code>
Fundamentos de finanzas y de la API	<code>docs/screener/GUIA_SEC_EDGAR_PARA_DEVS.md</code> ★ leer esto primero si las palabras "EPS" o "XBRL" no significan nada para vos

Próximo paso sugerido para un junior nuevo: leé `GUIA_SEC_EDGAR_PARA_DEVS.md` completo primero (son ~30 minutos y te da el vocabulario que esta guía asume que ya tenés). Después volvé a esta guía y seguila de nuevo de punta a punta — la segunda lectura se entiende mucho más rápido.